SLAM and Nav2 Simulation - Complete Manual

ROS 2 Humble | Gazebo Classic | slam_toolbox | AMCL | Nav2 | Packages: slam_sim and nav_sim

This manual explains everything behind the project. Part I covers all the theory: what each concept is, how each algorithm works, why we chose it and what the alternatives are. Part II explains the code of every file in both packages, line by line where it matters.

Part I - Theory

1. Project overview

The project simulates a mobile robot that first builds a map of an unknown building and then drives around it autonomously.

The robot is a differential-drive base with a 2D lidar and a depth camera. It lives in a Gazebo world made of four rooms connected by doorways, with obstacles inside the rooms.

The work is split into two stages and two packages:

- Stage 1, mapping (package slam_sim): a person drives the robot with the keyboard. The slam_toolbox node reads the lidar and the wheel odometry and builds a map. The map is saved to a file.
- Stage 2, navigation (package nav_sim): the robot loads the map, works out where it is (AMCL), and when the user clicks a goal in RViz2 it plans a path (NavFn) and follows it (Regulated Pure Pursuit or DWB) while avoiding obstacles.

The overall data flow:

```
Gazebo world + robot
   |-- /scan (lidar) ----------+
   |-- /odom + TF (wheels) ----+--> slam_toolbox --> /map  (Stage 1)
   |                           |
   |                           +--> AMCL on saved map --> map->odom TF
(Stage 2)
   |                                    |
   |                                    v
   |                          costmaps --> planner (NavFn) --> /plan
   |                                    |
   |                                    v
   |<---- /cmd_vel <---- controller (Pure Pursuit / DWB) <---- path
```

Why two stages? SLAM is computationally heavy and its map keeps changing. Navigation works best on a fixed, clean map with a light localization algorithm. Real robots are usually run this way: map once, then navigate many times. This project also supports navigation while mapping (slam:=true), which is useful in unknown places.

2. ROS 2 fundamentals used

Node. A single program that does one job, for example slam_toolbox, amcl or planner_server. A robot application is many nodes working together.

Topic. A named, one-way data stream. A publisher sends messages and any number of subscribers receive them. Examples: /scan (lidar), /odom (odometry), /cmd_vel (velocity command), /map.

Message. The data type sent on a topic, for example sensor_msgs/LaserScan, nav_msgs/Odometry, geometry_msgs/Twist, nav_msgs/OccupancyGrid.

Service. A request and reply call, used for quick operations. Example: the slam_toolbox serialize_map service saves the pose graph.

Action. A request that takes a long time and gives feedback along the way, and can be cancelled. An action has a goal, feedback and a result. Nav2 is built on actions: NavigateToPose (go to this pose), ComputePathToPose (plan a path), FollowPath (follow this path), Spin, BackUp and so on. RViz2 and the send_goals.py script both ask Nav2 to run the NavigateToPose action.

Parameter. A named setting of a node, loaded from a YAML file at start-up (for example max speed, map resolution). All tuning in this project is done through parameters.

Launch file. A Python script that starts and configures many nodes together, including passing parameters and launch arguments such as gui:=false.

Lifecycle node. A node with managed states: unconfigured, inactive, active. Nav2 nodes (map_server, amcl, planner_server and so on) are lifecycle nodes. A lifecycle_manager node brings them up in the right order and brings them down cleanly. When you run ros2 lifecycle get /amcl and it says active, that node is running normally.

QoS (Quality of Service). Rules for how a topic delivers data. Two settings matter here:

- Reliability: Reliable (resend lost messages) or Best Effort (fast, may lose messages). Sensor topics such as /scan use Best Effort.
- Durability: Volatile (only messages published after you subscribe) or Transient Local (the last message is kept and given to late subscribers). The /map topic is published once, so it uses Transient Local, and RViz2 must subscribe with Transient Local to see it.

A subscriber and publisher with incompatible QoS silently do not connect. This is a common reason for an empty display in RViz2.

Simulation time. Gazebo publishes its own clock on /clock. Nodes with use_sim_time set to true use that clock instead of the computer clock, so timestamps match the simulation even when the simulation runs slower than real time. Every node in this project sets use_sim_time to true.

Packages and workspaces. Code is organized into packages built with colcon inside a workspace (slam_ws). The build creates an install folder, and sourcing install/setup.bash makes the packages visible to ROS. Both of our packages are ament_cmake packages that contain only data files (launch, URDF, parameters), so nothing is compiled.

Plugins (pluginlib). Many Nav2 parts are chosen by name in a parameter file instead of being fixed in code. For example the global planner is the plugin nav2_navfn_planner/NavfnPlanner and the local controller is a plugin chosen under FollowPath. This is why we can change algorithms just by editing YAML.

DDS. The communication layer under ROS 2. It lets nodes find each other automatically on the network without a central master.

3. Simulation with Gazebo

3.1 What a simulator gives us

A simulator replaces the real world: it simulates physics (gravity, friction, collisions), the robot motors and the sensors. We can test SLAM and navigation without hardware, repeat experiments exactly and not damage anything.

3.2 Gazebo Classic

We use Gazebo Classic (version 11) through the gazebo_ros_pkgs package. It uses the ODE physics engine by default.

- The world file (rooms.world) is written in SDF (Simulation Description Format). It defines the ground, light, walls and obstacles.
- The robot is written in URDF/Xacro (Part II explains it) and inserted into the world by the spawn_entity.py program.
- Plugins are shared libraries loaded inside Gazebo that connect simulated hardware to ROS topics. We use three: libgazebo_ros_diff_drive.so (wheels and odometry), libgazebo_ros_ray_sensor.so (lidar) and libgazebo_ros_camera.so (depth camera).
- The Real Time Factor (RTF) shows how fast the simulation runs compared with real time. 1.0 is real time. A low RTF makes everything appear laggy and can break timing-sensitive parts such as TF and costmaps.

3.3 Why Gazebo Classic, and the alternatives

We chose it because it is the simulator that Humble tutorials, Nav2 examples and the TurtleBot3 packages were written for, it has ready-made plugins for diff drive, lidar and depth camera, and it runs on a normal laptop.

An honest note: Gazebo Classic reached end of life in January 2025. For new long-term projects the modern Gazebo (gz sim, for example Harmonic) with the ros_gz bridge is the recommended replacement, and the same URDF robot and the same SLAM and Nav2 configuration can be reused with small changes to the sensor and plugin sections.

Other simulators: NVIDIA Isaac Sim (photorealistic, GPU required, strong for cameras and AI), Webots (lightweight, cross-platform, good ROS 2 support), CoppeliaSim and Unity-based simulators.

3.4 URDF compared with SDF

URDF describes a robot only (links and joints) and is the standard format in ROS. SDF describes a whole world (robots, lights, physics) and is the Gazebo format. We use URDF for the robot because ROS tools such as robot_state_publisher and RViz2 read it, and SDF for the world. Gazebo extensions inside the URDF are written in gazebo tags.

4. Coordinate frames, TF and pose

A frame is a coordinate system attached to something. The pose of a robot is its position and orientation in a frame. In 2D that is x, y and heading (yaw).

ROS uses the right-handed convention of REP-103: x forward, y left, z up, and angles are counter-clockwise positive. Cameras use an optical convention: z forward, x right, y down. That is why we add camera_optical_link with a rotation.

4.1 The standard frames (REP-105)

- map: a fixed world frame. It does not drift, but the robot pose in it can jump when the estimate is corrected.
- odom: a frame that moves smoothly with wheel odometry. It never jumps, but it slowly drifts away from the truth.
- base_footprint: the robot, projected onto the ground. We use it as the main robot frame for SLAM and Nav2.
- base_link: the robot body, 7.5 cm above the ground in our model.
- lidar_link, camera_link, camera_optical_link, left_wheel, right_wheel: sensor and wheel frames.

The tree is:

```
map -> odom -> base_footprint -> base_link -> lidar_link
                                           -> camera_link ->
camera_optical_link
                                           -> left_wheel / right_wheel
```

4.2 Who publishes what

- odom to base_footprint: the Gazebo diff-drive plugin (from wheel odometry).
- map to odom: slam_toolbox while mapping, or AMCL while navigating. This transform is the correction. Wheel odometry says the robot is here in odom; the map algorithm says odom itself is shifted by this much relative to the map. Multiplying them gives the true pose in the map.
- The fixed transforms (base_link to lidar_link and so on): robot_state_publisher, from the URDF.
- The wheel transforms: the diff-drive plugin, with publish_wheel_tf set to true.

The reason for two separate frames: odometry is smooth and fast, which is good for control, while the map correction is slow and jumpy, which is good for planning. Keeping them apart gives both.

4.3 Timestamps and tolerance

TF stores transforms with timestamps. A node that receives a lidar scan at time t asks for the transform at time t. If the TF data is slightly late, the lookup fails (messages such as extrapolation error or Message Filter dropping message). A transform_tolerance parameter tells a node how long it may wait or how old a transform may be. We raised it for the costmaps because a slow simulation delays TF.

5. Differential drive and odometry

5.1 Kinematics

A differential-drive robot has two independently powered wheels on one axle plus passive casters. Let r be the wheel radius, L the distance between the wheels, vl and vr the wheel speeds at the ground.

```
forward speed   v = (vr + vl) / 2
turn rate       w = (vr - vl) / L

from a command (v, w):
  vr = v + w * L / 2
  vl = v - w * L / 2
  wheel angular speed = wheel linear speed / r
```

If both wheels turn the same way, the robot goes straight. If they turn in opposite directions, it rotates on the spot. The robot cannot move sideways. That limit is called a non-holonomic constraint, and it is why planners must produce paths a diff-drive robot can follow.

We chose a differential drive because it is the simplest robot to build and model, it can turn in place (so it can get out of tight spots), and every Nav2 default is designed for it.

5.2 Odometry

Odometry estimates pose by adding up small motions measured from the wheels:

```
x     = x     + v * cos(theta) * dt
y     = y     + v * sin(theta) * dt
theta = theta + w * dt
```

The diff-drive plugin computes this and publishes it on /odom and as the odom to base_footprint transform.

5.3 Why odometry drifts

Wheel slip, uneven floors, wheel size error and timing error all add small mistakes, and the mistakes add up forever. A one-degree heading error becomes a large position error after a few metres. A small heading error is the worst, because it rotates everything after it. This is the reason we need SLAM (to correct drift while building a map) and AMCL (to correct drift on a known map).

6. Sensors

6.1 The 2D lidar

Lidar (light detection and ranging) measures distance by firing a laser beam and timing the reflection (time of flight). A 2D lidar spins the beam in one plane and returns a ring of distances.

The data type is sensor_msgs/LaserScan. Its fields are the start and end angle, the angle step, the minimum and maximum range, and an array of distances (one per beam). Beams that hit nothing return infinity (or the maximum range).

Our simulated lidar: 360 beams over 360 degrees, 10 scans per second, range 0.12 to 10 m, Gaussian noise with a standard deviation of 1 cm, published on /scan in the lidar_link frame.

Why a 2D lidar is the main sensor: it gives accurate distances (centimetre level) directly, independent of lighting, in a small amount of data. That makes scan matching, map building and obstacle detection simple and fast. Its limits: it sees only one plane (low obstacles or overhanging tables are missed), and glass and mirrors confuse it.

6.2 The depth camera

A depth camera gives a distance for every pixel. Real ones work by stereo vision, structured light or time of flight. In Gazebo the depth sensor reads the distance from the renderer.

It publishes a colour image (/camera/image_raw), a depth image (/camera/depth/image_raw) and a 3D point cloud (/camera/points), all in the camera_optical_link frame. Our camera is 160 by 120 pixels at 10 Hz with a 60 degree field of view and a range of 0.3 to 8 m. The resolution is small on purpose: the sensor is rendered on the CPU and a larger image would slow the simulation.

Why the camera is not used for SLAM here: slam_toolbox and the 2D costmaps need a laser scan. The camera is included because many real robots carry one, and because it can be used for extensions. To use it for 2D navigation you convert it into a scan with the

depthimage_to_laserscan package, or use its point cloud in a 3D (voxel) costmap layer. It could also drive a visual SLAM system such as RTAB-Map.

6.3 Sensor noise

Real sensors are not perfect. Adding noise in simulation makes the algorithms face realistic conditions. A SLAM system that only works with perfect data is not useful.

7. The occupancy grid map

7.1 Definition

An occupancy grid divides the world into a regular grid of square cells. Each cell stores how likely it is that something solid is there. It is the standard 2D map for mobile robots and the map type used by slam_toolbox, AMCL and Nav2.

In ROS the message is nav_msgs/OccupancyGrid. It has a resolution (metres per cell, ours 0.05 m), a width and height in cells, the origin (the world pose of the cell at the lower-left corner) and a data array. Each value means:

- 0: free
- 100: occupied
- -1: unknown (never observed)
- 1 to 99: the probability, in percent, that the cell is occupied

7.2 How the map is built from scans

For every lidar beam, the cells along the beam, up to the hit point, are seen through, so they are likely free. The cell at the end of the beam is a hit, so it is likely occupied. Finding the cells along a beam is called ray casting (Bresenham line algorithm).

Evidence from many scans is combined with Bayes rule. A common way is the log-odds form, where each cell stores l = log(p / (1 - p)). Each observation adds a number to l (positive for a hit, negative for a pass-through), and the probability is recovered with p = 1 - 1 / (1 + e^l). Adding numbers is cheap and repeated evidence builds confidence: one noisy hit changes little, many hits make a cell clearly occupied.

slam_toolbox, which is based on Karto, keeps the stored scans and counts how many beams passed through and how many hit each cell, then thresholds the ratio.

7.3 The map file

A saved map is two files:

- A PGM image (rooms_map.pgm). One pixel is one cell. Black (0) is occupied, white (254) is free, grey (205) is unknown. Row 0 of the image is the top, which is the maximum y of the map.
- A YAML file (rooms_map.yaml) with the metadata:

```
image: rooms_map.pgm
mode: trinary
resolution: 0.05
origin: [-6.1, -6.1, 0.0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

- resolution: metres per pixel.
- origin: the pose (x, y, yaw) of the lower-left pixel in the map frame.
- negate: 0 means white is free and black is occupied.
- occupied_thresh and free_thresh: the map loader computes occupancy = (255 - pixel) / 255. Above occupied_thresh the cell is occupied, below free_thresh it is free, and in between it is unknown. In trinary mode each cell becomes exactly one of the three values.

7.4 Strengths, limits and alternatives

Strengths: simple, works directly with path planning and collision checks, constant-time cell lookup. Limits: it is 2D, assumes the world is static, and uses memory in proportion to the area.

Alternatives: 3D voxel maps such as OctoMap, signed-distance maps (TSDF and ESDF) used by drones and manipulators, feature or landmark maps (used by visual SLAM), point-cloud maps (LOAM, LIO-SAM for 3D lidar) and topological maps (graphs of places). For a ground robot in a flat building, the 2D occupancy grid is the best match for effort and accuracy.

8. SLAM - Simultaneous Localization and Mapping

8.1 The problem

To build a map the robot must know where it is when it takes each measurement. To know where it is, it needs a map. Neither is known at the start. SLAM solves both at the same time: it estimates the robot path and the map together.

The loop that runs continuously:

1. Predict: use odometry to guess how far the robot moved.
2. Observe: take a lidar scan.
3. Correct: align the new scan with what is already known to find the true pose, which removes odometry drift.
4. Update: add the scan, at the corrected pose, to the map.

8.2 Why SLAM is hard

- Drift: errors accumulate, so small mistakes early make the whole map bent.
- Data association: deciding whether the wall I see now is the same wall I saw ten minutes ago. A wrong decision tears the map.
- Loop closure: recognizing a revisited place and using it to fix the accumulated error.
- Perceptual aliasing: two different places that look the same (for example identical corridors).
- Real-time limit: the computation must keep up with the sensor.
- Dynamic world: people and moving objects break the assumption that the world is static.

8.3 Families of SLAM algorithms

Filter-based SLAM. Keep a probability distribution over the current pose (and sometimes the map) and update it at every step.

- EKF-SLAM: an extended Kalman filter over the robot pose and the landmark positions. Elegant but the cost grows with the square of the number of landmarks, and it needs identifiable landmarks. It is mostly historical for large maps.
- Particle-filter SLAM (FastSLAM, gmapping): each particle is one guess of the robot path and carries its own map. The Rao-Blackwellized particle filter in gmapping uses many particles. It works well for small areas but memory grows with the number of particles and the map size, and it cannot easily fix errors after the fact.

Graph-based SLAM. Keep all the poses and the measurements in a graph, and optimize the entire graph. This is the modern approach and it is what slam_toolbox uses. It is explained in 8.4.

Visual and visual-inertial SLAM. Use cameras (and an IMU). Examples: ORB-SLAM, VINS-Mono, RTAB-Map (appearance-based). They build feature maps or 3D maps and are good for drones and AR, but they do not directly give the 2D occupancy grid that Nav2 uses.

3D lidar SLAM. LOAM, LeGO-LOAM, LIO-SAM, FAST-LIO. They give accurate 3D point-cloud maps for vehicles and large outdoor robots.

8.4 Graph-based SLAM in detail

The system keeps a pose graph:

- Each node is a robot pose at the moment a scan was stored (a keyframe).
- Each edge is a constraint: the measured relative transform between two poses, together with how certain that measurement is. The information matrix holds the certainty.

SLAM then has two parts.

The front end builds the graph from raw data. It does scan matching and detects loop closures.

- Scan matching finds the transform that best overlaps two scans. Common methods are ICP (iterative closest point), correlative scan matching (try many candidate transforms inside a search window and score each) and NDT (normal distributions transform). Karto, and so slam_toolbox, uses correlative scan matching with a smoothed (Gaussian) lookup grid, so small noise does not ruin the score.
- Loop closure detection looks at old poses near the current pose. If a scan matches an old one with a high enough score, a new loop edge is added.

The back end optimizes the graph. It finds the poses that disagree least with all constraints, which means minimizing the sum over all edges of e transposed times the information matrix times e, where e is the difference between the pose-to-pose transform predicted by the current poses and the measured one. This is a non-linear least squares problem, solved with methods such as Gauss-Newton or Levenberg-Marquardt. slam_toolbox uses the Ceres solver library for this.

Why graph-based wins: after a loop closure the optimizer can spread the correction along the whole path and straighten the map, which a filter cannot do. The map is then rebuilt from the corrected poses.

8.5 Loop closure, in practice

While you drive down a hallway and around a block, odometry drift bends the path so the start and end do not meet. When the robot returns, the scan matches the first scans. The new edge pulls the end back to the start, and the optimizer redistributes the error over all poses. In our project, drive from the NW room through the doorways to the other rooms and back to the NW room, and watch the map snap into alignment.

9. slam_toolbox

9.1 What it is

slam_toolbox is a ROS 2 package for 2D lidar SLAM by Steve Macenski, the maintainer of Nav2. It is based on the Karto SLAM scan matcher and uses Ceres for graph optimization. It is the SLAM option recommended and tested with Nav2, and it is the one that is actively maintained in ROS 2.

Features:

- Synchronous and asynchronous mapping modes.
- Mapping mode (new map) and localization mode (use a saved map or pose graph).
- Lifelong mapping: keep refining a map over many sessions.
- Serialization: save and reload the full pose graph, not just the image, so mapping can be continued later.
- Loop closure and graph optimization.
- Interactive tools in RViz2 to edit the graph and map.
- Can handle large maps better than older particle-filter methods.

9.2 Synchronous versus asynchronous

- Synchronous (sync_slam_toolbox_node): processes every scan, one after another. No scan is dropped, but if the computer is slower than the lidar, a backlog builds up and the map lags.
- Asynchronous (async_slam_toolbox_node): processes the newest scan and skips scans if it is busy. It stays real-time on any hardware. We use async because the simulator is already heavy and we only need the map to keep up with a person driving.

9.3 How it processes a scan

1. A scan arrives with its odometry pose.
2. If the robot has not moved or turned enough since the last stored scan (minimum_travel_distance, minimum_travel_heading, both 0.2), the scan is ignored. This keeps the graph small.
3. The scan is matched against nearby stored scans (correlative scan matching) to correct the odometry pose.
4. The scan is added as a node, with edges to its neighbours.
5. Loop closure is tried (do_loop_closing, loop_search_maximum_distance).
6. If a loop closure was found, Ceres optimizes the graph and the poses move.

7. The occupancy grid is rebuilt and published on /map every map_update_interval seconds, and the map to odom transform is published.

9.4 Why slam_toolbox, and why not the others

- gmapping: a classic particle-filter SLAM. Light and simple, but memory grows with map size, it has no real loop-closure optimization, and the ROS 2 port is not actively maintained. Good for small teaching maps; slam_toolbox is more capable.
- Hector SLAM: needs no odometry and uses a high-rate lidar, which makes it good for handheld and drone use. It has no loop closure and degrades when the lidar rate is low or the scene has few features. We have wheel odometry and a 10 Hz lidar, so slam_toolbox, which uses odometry and has loop closure, suits us better.
- Cartographer (Google): very powerful, uses submaps and strong loop closure, and handles 2D and 3D. But configuration is complex (Lua files), tuning is hard and the ROS 2 package is less actively maintained.
- RTAB-Map: appearance-based graph SLAM with RGB-D, stereo and lidar support, making 3D and 2D maps. It is the best choice when the depth camera is the main sensor, but heavier, and unnecessary when we have a good lidar.
- ORB-SLAM, VINS: visual methods that make sparse feature maps, not occupancy grids.
- LOAM family: for 3D lidar.

We chose slam_toolbox because it fits our hardware (2D lidar plus wheel odometry), outputs the occupancy grid that Nav2 needs, handles loop closure with a real optimizer, is easy to configure with a single YAML file, runs in real time in async mode, and is the standard SLAM of the Nav2 project.

9.5 Key slam_toolbox parameters

- odom_frame, map_frame, base_frame: the TF frames (odom, map, base_footprint).
- scan_topic: /scan.
- mode: mapping.
- resolution: 0.05, the cell size of the map in metres.
- max_laser_range: 10.0, beams longer than this are ignored.
- minimum_travel_distance and minimum_travel_heading: how often keyframes are taken.
- map_update_interval: 2.0 seconds between map publications.
- do_loop_closing: true.
- loop_search_maximum_distance: 3.0, how far to look for loop candidates.

- loop_match_minimum_response_coarse and loop_match_minimum_response_fine: how good a match must be to accept a loop closure. Too low gives false closures that bend the map; too high misses real ones.
- correlation_search_space_dimension and the smear deviations: the search window and smoothness of scan matching.
- solver_plugin: the Ceres solver.

10. Localization and AMCL

10.1 Localization versus SLAM

Localization asks only: where am I in a map I already have? It is easier than SLAM and much lighter, because the map is fixed. Navigation uses it, so the robot does not need to run a full SLAM system every day.

There are three kinds of localization problem:

- Pose tracking: the start pose is known and we follow small changes.
- Global localization: the start pose is unknown and must be found anywhere on the map.
- Kidnapped robot: the robot is moved without being told and must realize it is lost and recover.

10.2 Bayes filters and Monte Carlo localization

A probabilistic localization method keeps a belief, a probability for every possible pose. Two repeating steps:

- Predict: move the belief according to the motion command and odometry.
- Update: re-weight the belief with the sensor measurement (poses that explain the scan well get a higher weight).

A Kalman filter does this with a single Gaussian, which works for tracking but cannot represent several different guesses. A particle filter represents the belief with many samples (particles), each one a possible pose. This handles any shape of belief and can handle global localization. This is Monte Carlo Localization (MCL).

10.3 AMCL - Adaptive Monte Carlo Localization

AMCL is the ROS and Nav2 implementation of MCL with adaptive particle count. It localizes a robot in a known 2D occupancy grid using a 2D laser scan and odometry.

Each cycle:

1. Predict. Every particle is moved with the odometry motion model plus random noise. The noise is set by alpha1 to alpha5. Larger alphas mean you trust the odometry less and the particles spread more.
2. Weight. For each particle, imagine where the lidar beams would end if the robot were at that pose, and compare them to the real scan. The better the match, the higher the weight. AMCL uses the likelihood field model: precompute for every map cell the distance to the nearest obstacle, then each beam endpoint is scored by that distance with a Gaussian (sigma_hit) plus a small random-noise term. It is fast because there is no ray tracing per particle.
3. Resample. Draw a new set of particles in proportion to the weights. Particles in wrong places die out, and good ones are duplicated.
4. Estimate. The robot pose is the weighted mean of the particles (of the biggest cluster), published as the map to odom transform.

Adaptive means the number of particles changes between min_particles (500) and max_particles (2000). AMCL uses KLD sampling: when the particles are spread out (uncertain), it uses more; when they cluster tightly (confident), it uses fewer, which saves CPU.

AMCL only updates when the robot has moved enough (update_min_d 0.25 m or update_min_a 0.2 rad). This avoids noise accumulation while standing still.

10.4 Key AMCL parameters in this project

- base_frame_id, odom_frame_id, global_frame_id: base_footprint, odom, map.
- scan_topic: scan.
- robot_model_type: nav2_amcl::DifferentialMotionModel.
- laser_model_type: likelihood_field.
- min_particles, max_particles: 500 and 2000.
- laser_max_range: 10.0 and max_beams: 60 (only 60 beams are used, to save CPU).
- sigma_hit: 0.2, z_hit: 0.5, z_rand: 0.5: sensor model weights.
- alpha1 to alpha5: 0.2, the odometry noise.
- set_initial_pose: true and initial_pose (-3.0, 1.5, yaw 0): the known spawn pose, so we do not have to click it.
- transform_tolerance: 1.0.

10.5 Why AMCL, and the alternatives

Why AMCL:

- It is the default, mature localization of Nav2, tested for years.

- It needs only a 2D lidar, wheel odometry and a saved map, which we have.
- It is light on the CPU.
- It can start with a rough pose and recover from small errors.

Alternatives:

- slam_toolbox in localization mode: matches scans against a saved pose graph. It can be more accurate in large or featureless maps and keeps updating the map. It is heavier than AMCL, and it needs the serialized pose graph instead of a plain image map. A good option for large indoor spaces.
- Cartographer in pure localization mode: similar idea, using its submaps.
- EKF localization with robot_localization: fuses odometry, IMU and GPS into a smooth estimate. It does not use a map, so it gives the odom frame but not the map correction. In real robots it is often used together with AMCL, for the odom to base transform.
- NDT or ICP-based localization (for example for 3D lidar maps): needed in 3D.
- GPS, UWB beacons, AprilTags: used outdoors or when infrastructure exists.
- Visual localization: matching camera images to a visual map.

If you do not have a map and want to move at once, you use SLAM (slam:=true in this project). If you have a map, you use AMCL. This is why nav_sim supports both.

10.6 Common AMCL problems

- Wrong initial pose: the particles never converge, or the robot model does not line up with the scan in RViz2. Fix it with 2D Pose Estimate.
- Symmetric maps: our world has four similar rooms, so global localization from nothing can pick the wrong room. That is why we give AMCL the initial pose.
- Slow simulation: late TF and scans make AMCL drop messages.

11. Nav2 - the navigation framework

11.1 What Nav2 is

Nav2 (Navigation 2) is the standard ROS 2 navigation stack. It takes a goal pose and drives a robot there safely: it localizes, plans a path, follows it, avoids obstacles and recovers when stuck. It is the ROS 2 successor of the ROS 1 navigation stack (move_base).

Why Nav2 rather than writing our own or using something else:

- It is the only actively maintained navigation framework for ROS 2 mobile robots, with a large community and many tested plugins.

- It is modular: planner, controller, localizer and recovery behaviours are separate servers that can be swapped by editing a parameter file.
- It handles the many details that are easy to get wrong in a custom node: transform timing, costmaps, goal checking, cancelling, recovery, lifecycle management.
- The ROS 1 move_base is not available in ROS 2, and MoveIt is for robot arms, not mobile bases.

11.2 The servers

Nav2 is a set of cooperating servers (nodes). Most are lifecycle nodes and talk through actions and topics.

- bt_navigator: the brain. It receives the NavigateToPose action goal and runs a behavior tree that decides when to plan, follow, replan or recover.
- planner_server: computes the global path (ComputePathToPose action). It owns the global costmap.
- controller_server: follows the path (FollowPath action) and produces velocity commands. It owns the local costmap.
- smoother_server: optionally smooths a path.
- behavior_server: recovery behaviours: spin, backup, drive on heading, wait.
- waypoint_follower: drives through a list of waypoints.
- velocity_smoother: limits acceleration on the velocity commands.
- map_server: loads and serves the saved map on /map.
- amcl: localization (Section 10).
- lifecycle_manager: starts and monitors all of the above. In this project there are two: one for localization (map_server, amcl) and one for navigation (the rest).

11.3 What happens when you send a goal

1. RViz2 (2D Goal Pose) publishes /goal_pose, or a script calls the NavigateToPose action.
2. bt_navigator starts its behavior tree.
3. The tree calls planner_server, which plans a path on the global costmap and publishes it on /plan (the green line).
4. The tree calls controller_server with that path. The controller runs at a fixed rate (10 Hz here): it reads the robot pose and the local costmap, then outputs a velocity command on /cmd_vel_nav.
5. velocity_smoother limits acceleration and publishes /cmd_vel.
6. The Gazebo diff-drive plugin turns /cmd_vel into wheel motion.
7. Meanwhile the tree replans (once per second) to adapt to new obstacles.

8. If progress stops, the tree clears the costmaps and runs recovery behaviours.
9. When the goal checker says the robot is within tolerance, the action succeeds.

11.4 Which Nav2 components this project uses

- Localization: AMCL (or slam_toolbox in SLAM mode).
- Global planner: NavFn.
- Local controller: Regulated Pure Pursuit (default), DWB (alternative).
- Costmap layers: static layer, obstacle layer, inflation layer.
- Recovery behaviours: spin, backup, drive on heading, wait.
- Goal checker: SimpleGoalChecker. Progress checker: SimpleProgressChecker.
- Velocity smoother, path smoother (SimpleSmoother, configured, available to custom trees) and waypoint follower.
- Behavior tree: the default NavigateToPose tree with replanning and recovery.

12. Behavior trees

A behavior tree (BT) is a way to organize decisions as a tree of small nodes. It is easier to read and change than a state machine. Nav2 uses BehaviorTree.CPP.

Every node returns SUCCESS, FAILURE or RUNNING. Node types:

- Action nodes do something: ComputePathToPose, FollowPath, Spin, BackUp, Wait, ClearEntireCostmap.
- Condition nodes check something: GoalReached, IsStuck, TransformAvailable.
- Control nodes decide which children to run: Sequence (run children in order until one fails), Fallback (try children until one succeeds), RecoveryNode (run a child, and on failure run a recovery then retry), PipelineSequence (run in a pipeline, so planning and following run together), RoundRobin (try each recovery in turn).
- Decorator nodes change the behavior of one child: RateController (run at most N times per second), DistanceController, GoalUpdater.

The default tree used here (navigate to pose with replanning and recovery) in plain words:

```
Navigate with recovery (retry up to 6 times)
  Navigate:
    PipelineSequence:
      every 1 second: ComputePathToPose  (replan)
      FollowPath                         (drive)
    On failure, recovery actions:
      clear the local and global costmaps
      then round-robin: Spin, Wait, BackUp
```

This is what you see when the robot is stuck: it spins on the spot, waits or backs up, then tries again. If all retries fail the goal is aborted. If the robot moves a little, rotates and then stops, it usually means the controller failed once and the recovery spin ran, and then the goal was aborted.

Why behavior trees: the logic is stored in an XML file and can be customized (for example go to a charging station when the battery is low) without changing C++ code.

13. Costmaps

13.1 Why a costmap

The occupancy grid only says what is a wall. A robot has size, and it should not graze walls. A costmap turns the map and sensors into a grid of costs that a planner can use directly: cells are free, dangerous or forbidden depending on how close they are to obstacles and how big the robot is.

Nav2 uses the costmap_2d package. A costmap is built from layers, and each layer adds its information on top.

13.2 Cost values

Each cell has a value from 0 to 255:

- 0: free space.
- 1 to 252: increasing cost, as the cell gets closer to an obstacle. Planners avoid high costs but may use them.
- 253: inscribed. The robot centre is inside the radius of the robot circle from an obstacle, so the robot would certainly touch it.
- 254: lethal. The cell is an obstacle.
- 255: no information (unknown).

13.3 Layers used here

- Static layer: copies the saved map (or the slam_toolbox map) into the costmap. It gives the permanent walls. It is used in the global costmap only.
- Obstacle layer: marks cells hit by the lidar as lethal and clears cells along beams that pass through them (ray tracing). It adds new and moving obstacles that are not on the static map. Used in both costmaps with /scan as the source.
- Inflation layer: spreads cost around every lethal cell so the robot keeps a safe distance. Our settings are cost_scaling_factor 3.0 and inflation_radius 0.55 m.

- Other layers that exist and are not used here: the voxel layer (3D obstacles from point clouds or depth cameras), the range layer (sonar or IR), and the keepout and speed filters (zones where the robot must not go or must go slowly).

13.4 Inflation formula

For a cell at distance d from the nearest obstacle:

```
d = 0                          cost = 254 (lethal)
0 < d <= inscribed radius      cost = 253 (inscribed)
d > inscribed radius           cost = 252 * exp(-cost_scaling_factor * (d -
inscribed_radius))
d > inflation_radius           cost = 0
```

A bigger cost_scaling_factor makes the cost fall faster, so the robot may drive closer to walls. A bigger inflation_radius makes it keep a larger gap. Both must be balanced with the doorways: our doorways are 1.4 m wide and the robot radius is 0.25 m, so a clear corridor of 0.45 m remains on each side of the door centre line. An inflation radius that is too big can make the robot think a door is blocked.

13.5 Robot footprint

The costmap needs the robot shape. We use robot_radius 0.25 m, which is a circle around the robot. The body is 0.40 by 0.30 m, and the farthest corner is 0.25 m from the centre, so the circle covers the whole robot. A circle is faster to check than a polygon and is safe for a robot that can turn on the spot. The footprint can also be a polygon list.

13.6 Global and local costmaps

- Global costmap: covers the whole map. Frame is map. Layers: static, obstacle, inflation. It is updated at 1 Hz. The global planner plans on it.
- Local costmap: a small window (3 by 3 m) that moves with the robot (rolling window). Frame is odom, which is smooth and so good for control. Layers: obstacle, inflation. It is updated at 5 Hz. The controller uses it for fast obstacle avoidance.

Why two: planning needs the complete picture but does not need to be fast. Control needs to be fast but only needs to see nearby. Using the odom frame for the local costmap avoids jumps when AMCL corrects the pose.

13.7 Unknown space

track_unknown_space (true for the global costmap) keeps cells that were never seen as unknown, and the planner option allow_unknown (True) lets paths cross unknown cells. In our map there are no unknown cells inside the arena.

14. Global planners

14.1 The job

The global planner finds a route from the current pose to the goal over the global costmap, avoiding lethal cells and preferring low-cost cells. It runs once per replanning cycle and looks at the whole map. It does not drive; it returns a list of poses (nav_msgs/Path).

14.2 Search algorithms behind planners

- Dijkstra: expands outward from the start in order of cost until it reaches the goal. It always finds the cheapest path. It explores in all directions, so it is slower on big maps.
- A* (A star): like Dijkstra but guided by a heuristic (an estimate of the remaining distance to the goal), so it explores fewer cells and is faster. It still returns the cheapest path when the heuristic is admissible.
- Sampling-based planners (RRT, PRM): good for high-dimensional spaces such as robot arms. Not used for 2D grids.

14.3 NavFn - the planner we use

NavFn is the classic Nav2 global planner. It computes a navigation function: a potential field over the grid in which the value of each cell is the cost to reach the goal. It uses Dijkstra by default (use_astar False). The path is then traced by following the gradient downhill from the start to the goal.

Parameters used: tolerance 0.5 (the path may end up to 0.5 m from the goal if the exact cell is not reachable), use_astar False, allow_unknown True.

Why NavFn:

- It is simple, fast and reliable on a small indoor grid.
- A differential-drive robot can turn on the spot, so we do not need a planner that respects turning-radius limits.
- It works with any costmap and needs almost no tuning.

Weaknesses: it ignores the robot kinematics, so paths can be slightly jagged (the grid directions), and the path may hug corners. The local controller and, if added, the smoother take care of these.

14.4 Alternatives

- Smac Planner 2D: A* on the grid with 8-connected moves, smoother and faster than NavFn, a good drop-in replacement.

- Smac Hybrid-A*: plans in continuous (x, y, heading) space with a minimum turning radius. Required for car-like (Ackermann) robots and good for large diff-drive or legged robots that should not turn in place.
- Smac State Lattice: uses pre-computed feasible motion primitives. Needed for non-circular robots and non-holonomic robots needing exact feasible paths.
- Theta*: any-angle planner that makes straighter paths on grids.

We use NavFn because the task was to use a simple global planner. If you want smoother paths, switch the plugin to Smac Planner 2D in the planner_server section.

14.5 Path smoothing

The smoother_server can smooth the planner output. We configured the SimpleSmoother (tolerance, max_its, do_refinement), which pushes path points to reduce sharp angles. The default behavior tree does not call it, so it is available but not active unless you use a custom tree.

15. Local planners (controllers)

15.1 The job

The global planner draws a route on a static picture. The local planner (called the controller in Nav2) is the part that actually drives. It runs at 10 Hz here. Every cycle it looks at where the robot is, the global path ahead and the local costmap, and decides the next linear and angular velocity. It reacts to things the global plan could not know, such as a box that was added or a moving person, and it makes the motion respect the robot limits.

15.2 DWB - the Dynamic Window approach

DWB is the Nav2 version of the Dynamic Window Approach (Fox, Burgard and Thrun, 1997). It works by simulating many short trajectories and choosing the best one.

Each cycle:

1. Compute the dynamic window: the set of velocity pairs (v, w) the robot can reach in the next time step, given its current velocity and its acceleration limits.
2. Sample that window (vx_samples 10 by vtheta_samples 20 in our file).
3. For each sample, simulate the robot driving at that velocity for sim_time seconds (2.0) to get a trajectory.
4. Score each trajectory with a set of critics. Each critic returns a cost and has a scale (weight). The total score is the weighted sum.
5. Pick the trajectory with the lowest cost and send its first velocity.

The critics we configured:

- BaseObstacle: penalizes trajectories that go through high-cost cells of the local costmap.
- PathAlign: penalizes trajectories whose end is not aligned with the global path.
- PathDist: penalizes trajectories that end far from the global path.
- GoalAlign: penalizes not pointing at the goal when approaching it.
- GoalDist: penalizes trajectories that end far from the goal.
- RotateToGoal: when close to the goal, rotates to the goal heading.
- Oscillation: penalizes switching back and forth, which prevents jitter.

Strengths: handles obstacles flexibly, and it is general. Weaknesses: many parameters to tune (critic weights fight each other), more CPU use, and with poor tuning the robot can hesitate, oscillate or stop, which is why we kept it as an alternative.

15.3 Regulated Pure Pursuit - our default

Pure pursuit is a very old and simple path-tracking method. Choose a lookahead point on the path at distance L in front of the robot. Then drive an arc that passes through that point.

Geometry: if the lookahead point is at lateral offset y in the robot frame, the curvature of the circle through it is

```
curvature = 2 * y / L^2
angular velocity w = v * curvature
```

So the robot steers toward the point. A short lookahead follows the path tightly but may oscillate; a long lookahead is smooth but cuts corners.

Regulated Pure Pursuit (RPP) adds practical features:

- Regulation: slows down on tight curves (regulated_linear_scaling_min_radius 0.9, min speed 0.15) and when close to obstacles.
- Collision detection: it projects the arc forward a short time (max_allowed_time_to_collision_up_to_carrot 1.0 s) against the local costmap and stops if it would hit something.
- Rotate to heading: when the path heading differs from the robot heading by more than rotate_to_heading_min_angle (0.785 rad, 45 degrees) it first turns on the spot at 0.8 rad/s. This is ideal for a differential-drive robot.
- Approach slow-down: slows near the goal (approach_velocity_scaling_dist 0.6).

Our parameters: desired_linear_vel 0.25 m/s, lookahead_dist 0.6 m (range 0.3 to 0.9 m), lookahead_time 1.5 s.

Why it is the default: it needs very few parameters, behaves smoothly and predictably, and was designed for differential-drive indoor robots. We started with DWB and the robot moved a little, rotated and stopped, so we changed the default to RPP and kept DWB as an alternative file (nav2_params_dwb.yaml).

15.4 Other controllers

- MPPI (Model Predictive Path Integral): samples thousands of trajectories with a motion model, picks the best by a weighted combination. Very flexible and smooth, supports differential, omni and Ackermann robots. Heavy on the CPU or GPU.
- TEB (Time Elastic Band): optimizes a trajectory in time and space with kinematic limits. Good for car-like robots and tight places, but the ROS 2 port is community maintained.
- Rotation Shim: a wrapper that rotates the robot toward the path before handing control to another controller.

16. Other Nav2 pieces used

16.1 Goal checker

Decides when the robot has arrived. SimpleGoalChecker: position within xy_goal_tolerance (0.15 m) and heading within yaw_goal_tolerance (0.25 rad). With stateful True, once the position is reached it only checks the heading. Too tight tolerances make the robot circle near the goal.

16.2 Progress checker

Decides if the robot is stuck. SimpleProgressChecker: the robot must move at least required_movement_radius (0.2 m) within movement_time_allowance (20 s). If not, the controller fails and recovery starts.

16.3 Controller failure tolerance

failure_tolerance (1.0 s) is how long the controller may fail to produce a valid command before the action is considered failed. A short value aborts on any hiccup; a longer value tolerates short glitches.

16.4 Recovery behaviours (behavior_server)

- Spin: rotates in place to clear the surroundings and refresh the sensors.
- BackUp: drives back a short distance.
- DriveOnHeading: drives forward on the current heading.
- Wait: stands still for a while.

Parameters: max_rotational_vel 1.0, min_rotational_vel 0.4, rotational_acc_lim 3.2, simulate_ahead_time 2.0 (so the behavior checks for collision before it moves).

16.5 Velocity smoother

Takes the controller output (/cmd_vel_nav) and limits acceleration and speed before sending /cmd_vel. In our file: max_velocity [0.25, 0, 0.8], max_accel [1.0, 0, 2.5], max_decel [-1.0, 0, -2.5], feedback OPEN_LOOP (it assumes the robot did what was commanded). This prevents jerky starts that could make wheels slip or overturn a robot.

16.6 Waypoint follower and Simple Commander

The waypoint_follower runs a list of goals in order. nav2_simple_commander is a Python API (BasicNavigator) that wraps the Nav2 actions: goToPose, followWaypoints, isTaskComplete, getFeedback, getResult. Our send_goals.py uses it to tour the four rooms.

16.7 Lifecycle manager

Brings Nav2 nodes through configure and activate in the right order, and with autostart true does it automatically. If one node fails, it can bring the group down. When something does not work, check ros2 lifecycle get for each node.

17. End to end: what happens at runtime

17.1 Stage 1 - mapping (slam_sim)

1. The launch file starts Gazebo with rooms.world and spawns the robot at (-3.0, 1.5).
2. robot_state_publisher publishes the fixed transforms from the URDF.
3. Gazebo plugins publish /scan, /odom, the odom to base_footprint transform, and the camera topics.
4. After 8 seconds slam_toolbox starts. The first scan creates the first node of the graph and the first map.
5. You drive with the keyboard. /cmd_vel goes to the diff-drive plugin, the wheels turn, odometry changes.
6. slam_toolbox matches scans, adds nodes and edges, optimizes after loop closures, and publishes /map and map to odom.
7. RViz2 shows the live map. You save it with map_saver_cli.

17.2 Stage 2 - navigation (nav_sim)

1. The launch file includes slam_sim sim.launch.py (Gazebo and the robot) without its RViz2.
2. After 12 seconds the localization group starts: map_server loads rooms_map.yaml and AMCL starts.
3. The navigation group starts: planner_server, controller_server, smoother_server, behavior_server, bt_navigator, waypoint_follower, velocity_smoother. Both costmaps start.
4. After 22 seconds a message on /initialpose gives AMCL the start pose (-3.0, 1.5). AMCL creates its particles and publishes map to odom. The costmaps now have all the transforms they need.
5. You click 2D Goal Pose. bt_navigator runs the tree: NavFn plans a path (green line), the controller follows it, the velocity smoother limits acceleration, the wheels move.
6. AMCL keeps correcting drift every 0.25 m of movement.
7. When the goal is reached the action finishes with SUCCEEDED.

In SLAM mode (slam:=true) steps 2 and 4 are replaced by slam_toolbox, which provides both the map and map to odom.

18. Why each choice - summary

- ROS 2 Humble: a long-term support release (supported until May 2027) with the best Nav2 and slam_toolbox packages and the most documentation. Alternatives: Jazzy and later, which need a newer Ubuntu and a different simulator setup.
- Gazebo Classic: mature plugins for diff drive, lidar and depth camera, and works with the Humble tutorials. It is end of life, so for new work consider Gazebo Harmonic.
- Differential drive: simplest robot, can turn on the spot, default for Nav2. Alternatives: Ackermann (car-like), omnidirectional (mecanum), legged.
- 2D lidar as the main sensor: accurate, light, lighting independent, direct input for scan matching and costmaps. Alternative: depth camera or 3D lidar.
- Depth camera included but unused for SLAM: shows a typical real sensor set and can be added to the costmap or used for visual SLAM later.
- Occupancy grid map: the standard 2D format of slam_toolbox, AMCL and Nav2. Alternatives: OctoMap, TSDF, feature maps.
- slam_toolbox in async mode: graph-based with real loop closure, active maintenance, Nav2 standard, real-time on slow hardware.
- AMCL: light, mature, needs only lidar, odometry and a map. Alternatives: slam_toolbox localization mode, Cartographer localization, NDT.

- NavFn global planner: simple, fast, reliable on a small grid for a robot that can turn in place. Alternatives: Smac 2D, Hybrid-A*, State Lattice, Theta*.
- Regulated Pure Pursuit local controller: few parameters, smooth, built for diff drive. Alternatives: DWB (kept as an option), MPPI, TEB.
- Two costmaps: global (whole map, 1 Hz) for planning and local (3 by 3 m rolling window in the odom frame, 5 Hz) for fast avoidance.
- Default behavior tree: standard replanning and recovery, no need for a custom tree.

19. Glossary

- AMCL: Adaptive Monte Carlo Localization, a particle-filter localization on a known map.
- Action: a ROS 2 long-running request with feedback and a result.
- Behavior tree: a tree of small nodes that decides what the robot should do next.
- Costmap: a grid of traversal costs built from the map and sensors, with obstacles inflated.
- Differential drive: two independently driven wheels, steering by speed difference.
- DWB: Dynamic Window approach local planner in Nav2.
- Frame: a coordinate system attached to something (map, odom, base_footprint).
- Global planner: finds the full route from start to goal.
- Graph SLAM: SLAM that stores poses and constraints as a graph and optimizes it.
- Inflation: raising the cost around obstacles to keep a safety distance.
- Lidar: a sensor that measures distances with laser light.
- Lifecycle node: a node with managed states (unconfigured, inactive, active).
- Local planner (controller): follows the global path and avoids nearby obstacles.
- Loop closure: recognizing a revisited place to remove accumulated drift.
- Lethal cell: a costmap cell that is an obstacle (cost 254).
- Occupancy grid: a grid map where each cell holds the probability of being occupied.
- Odometry: estimating motion by adding up wheel movements.
- Particle filter: a method that tracks a belief with many sample guesses.
- Plugin: a swappable component chosen by name in a parameter file.
- Pose: position and orientation of a robot in a frame.
- QoS: Quality of Service, rules about reliability and durability of topics.
- Real Time Factor: how fast the simulation runs compared to real time.
- Recovery behavior: an action such as spin or backup used when the robot is stuck.
- Resolution: the size of one map cell in metres.

- RPP: Regulated Pure Pursuit controller.
- Scan matching: finding the transform that best aligns two scans.
- SLAM: Simultaneous Localization and Mapping.
- TF: the ROS 2 transform system that tracks all frames.
- URDF and Xacro: robot description formats, Xacro adds macros and maths.
- use_sim_time: makes nodes use the simulator clock.

Part II - Code explanation

This part walks through every file. The workspace is slam_ws with two packages under src: slam_sim (robot, world, SLAM) and nav_sim (Nav2).

20. Build files

20.1 package.xml

Every ROS 2 package has a package.xml. It names the package, gives the maintainer and licence, and lists dependencies.

- buildtool_depend ament_cmake: the build system used.
- exec_depend entries: packages needed at run time. slam_sim needs gazebo_ros, gazebo_ros_pkgs, robot_state_publisher, xacro, slam_toolbox, rviz2, teleop_twist_keyboard and nav2_map_server. nav_sim needs slam_sim, nav2_bringup, nav2_amcl, nav2_map_server, nav2_navfn_planner, dwb_core, nav2_regulated_pure_pursuit_controller, nav2_simple_commander and rviz2.
- export build_type ament_cmake: tells colcon how to build.

With rosdep these dependencies can be installed automatically.

20.2 CMakeLists.txt

Both packages contain only data files, so CMake only installs folders:

```
cmake_minimum_required(VERSION 3.8)
project(slam_sim)
find_package(ament_cmake REQUIRED)
install(DIRECTORY launch urdf worlds config rviz DESTINATION
share/${PROJECT_NAME})
ament_package()
```

The install(DIRECTORY ...) line copies the folders into install/slam_sim/share/slam_sim, which is where get_package_share_directory looks. With colcon build --symlink-install these are symbolic links, so editing a source file takes effect at once without rebuilding.

nav_sim has the same layout, plus rviz and maps in the folder list, and one extra line:

```
install(PROGRAMS scripts/send_goals.py DESTINATION lib/${PROJECT_NAME})
```

This installs the script as an executable, which is what makes ros2 run nav_sim send_goals.py work.

21. slam_sim/urdf/robot.urdf.xacro

This file describes the robot body, wheels, sensors and the Gazebo plugins. Xacro is URDF with properties, math and macros. At launch it is converted into plain URDF by the xacro command.

21.1 Properties

```
<xacro:property name="PI" value="3.14159265"/>
<xacro:property name="body_l" value="0.40"/>
<xacro:property name="body_w" value="0.30"/>
<xacro:property name="body_h" value="0.10"/>
<xacro:property name="wheel_r" value="0.05"/>
<xacro:property name="wheel_w" value="0.03"/>
<xacro:property name="wheel_sep" value="0.34"/>
```

Properties are named constants, used later as ${name}, and expressions such as ${wheel_sep/2} are computed. Change a value here and the whole robot changes consistently. wheel_sep (L) must stay larger than body_w (0.30) so the wheels sit outside the body.

21.2 The inertia macro

```
<xacro:macro name="box_inertia" params="m x y z">
  <inertial>
    <mass value="${m}"/>
    <inertia ixx="${m*(y*y+z*z)/12}" ixy="0" ixz="0"
             iyy="${m*(x*x+z*z)/12}" iyz="0" izz="${m*(x*x+y*y)/12}"/>
  </inertial>
</xacro:macro>
```

Gazebo needs mass and the inertia tensor of every link, or the physics behaves wrongly. For a solid box of mass m and sides x, y, z, the moments of inertia are m times the sum of the squares of the two other sides divided by 12. The macro computes this so we do not write the numbers by hand.

21.3 The frames and the body

```
<link name="base_footprint"/>
<joint name="base_footprint_joint" type="fixed">
  <parent link="base_footprint"/><child link="base_link"/>
  <origin xyz="0 0 0.075"/>
</joint>
<link name="base_link"> ... box 0.40 x 0.30 x 0.10, mass 5.0 ... </link>
```

base_footprint is an empty link (no geometry) on the ground. base_link is the body, 0.075 m above it. The 0.075 m is chosen so the wheels, whose centres are 0.025 m below base_link, touch the ground: 0.075 - 0.025 = 0.05 = the wheel radius.

Each link has a visual (what you see), a collision (what the physics uses) and an inertial (mass and inertia). Here the visual and collision use the same box.

21.4 The wheel macro

```
<xacro:macro name="wheel" params="name y">
  <link name="${name}_wheel">
    <visual>
      <origin rpy="${PI/2} 0 0"/>
      <geometry><cylinder radius="${wheel_r}" length="${wheel_w}"/>
</geometry>
    </visual>
    <visual name="stripe"> ... a white box on the rim ... </visual>
    <collision> ... same cylinder ... </collision>
    <inertial> ... mass 0.5 ... </inertial>
  </link>
  <joint name="${name}_wheel_joint" type="continuous">
    <parent link="base_link"/><child link="${name}_wheel"/>
    <origin xyz="0 ${y} -0.025"/>
    <axis xyz="0 1 0"/>
  </joint>
  <gazebo reference="${name}_wheel">
    <mu1>1.0</mu1><mu2>1.0</mu2><kp>500000.0</kp><kd>10.0</kd>
  </gazebo>
```

```
</xacro:macro>

<xacro:wheel name="left"  y="${wheel_sep/2}"/>
<xacro:wheel name="right" y="${-wheel_sep/2}"/>
```

- A cylinder in URDF has its axis along z. Rotating it by 90 degrees about x (rpy PI/2 0 0) lays it on its side so its axis is along y, the axle direction.
- The joint is continuous, which means unlimited rotation. The axis 0 1 0 means it spins about y. Positive rotation drives the robot forward.
- The joints are at y = +0.17 (left) and y = -0.17 (right).
- The white stripe is only a visual. A plain cylinder looks the same while spinning, so without it you cannot see the wheel turn in Gazebo.
- mu1 and mu2 are friction coefficients in the two directions. 1.0 gives grip, so the wheels push the robot instead of slipping. kp and kd are the contact stiffness and damping.
- The macro is called twice, so left and right wheels are identical but mirrored.

21.5 The casters

```
<xacro:macro name="caster" params="name x">
  <link name="${name}_caster"> ... sphere radius 0.025, mass 0.1 ... </link>
  <joint name="${name}_caster_joint" type="fixed">
    <parent link="base_link"/><child link="${name}_caster"/>
    <origin xyz="${x} 0 -0.05"/>
  </joint>
  <gazebo reference="${name}_caster"><mu1>0.0</mu1><mu2>0.0</mu2></gazebo>
</xacro:macro>
<xacro:caster name="front" x="0.15"/>
<xacro:caster name="rear"  x="-0.15"/>
```

Two drive wheels cannot hold a robot upright, so two small spheres at the front and rear keep it level. They are fixed (not rotating), with zero friction so they slide and do not resist turning. Their centres are 0.05 m below base_link (0.025 m above the ground), and the radius 0.025 m makes them touch the ground.

21.6 The sensor links

```
<joint name="lidar_joint" type="fixed">
  <parent link="base_link"/><child link="lidar_link"/>
  <origin xyz="0.05 0 0.075"/>
</joint>
<joint name="camera_joint" type="fixed">
  <parent link="base_link"/><child link="camera_link"/>
  <origin xyz="0.2 0 0.03"/>
</joint>
<link name="camera_optical_link"/>
<joint name="camera_optical_joint" type="fixed">
  <parent link="camera_link"/><child link="camera_optical_link"/>
  <origin rpy="${-PI/2} 0 ${-PI/2}"/>
</joint>
```

- The lidar is a small red cylinder 5 cm in front of the centre, 7.5 cm above base_link. That places the scan plane at 0.15 m above the ground, above the body (top at 0.125 m) and the camera (top at 0.12 m), so the robot does not see itself, and below the walls and obstacles, which are at least 0.8 m tall.
- The camera is a small box at the front. Its optical frame is rotated by -90 degrees about x and then -90 degrees about z, turning the ROS body convention (x forward, y left, z up) into the optical convention (z forward, x right, y down).

21.7 Gazebo materials

```
<gazebo reference="base_link"><material>Gazebo/Blue</material></gazebo>
<gazebo reference="left_wheel"><material>Gazebo/Black</material></gazebo>
```

Gazebo ignores the colours in the URDF visual tags, so colours are given again with Gazebo material names. Without these the robot is white or grey.

21.8 The differential-drive plugin

```
<plugin name="diff_drive" filename="libgazebo_ros_diff_drive.so">
  <update_rate>50</update_rate>
  <left_joint>left_wheel_joint</left_joint>
  <right_joint>right_wheel_joint</right_joint>
  <wheel_separation>${wheel_sep}</wheel_separation>
  <wheel_diameter>${2*wheel_r}</wheel_diameter>
  <max_wheel_torque>20</max_wheel_torque>
  <max_wheel_acceleration>2.0</max_wheel_acceleration>
  <command_topic>cmd_vel</command_topic>
  <odometry_topic>odom</odometry_topic>
  <odometry_frame>odom</odometry_frame>
  <robot_base_frame>base_footprint</robot_base_frame>
  <publish_odom>true</publish_odom>
  <publish_odom_tf>true</publish_odom_tf>
  <publish_wheel_tf>true</publish_wheel_tf>
</plugin>
```

This single plugin is the robot motor controller and odometry.

- It subscribes to /cmd_vel (linear and angular velocity), converts it to left and right wheel speeds with the formulas from Section 5.1 and drives the two joints.
- wheel_separation and wheel_diameter must match the model, otherwise the odometry is wrong and the turn rates are wrong.
- max_wheel_torque and max_wheel_acceleration limit what the simulated motors can do.
- It publishes /odom and the odom to base_footprint transform (publish_odom_tf), at 50 Hz.
- publish_wheel_tf publishes the wheel joint transforms, so no joint_state_publisher is needed and the wheels turn in RViz2.

21.9 The lidar sensor

```
<sensor name="lidar" type="ray">
  <update_rate>10</update_rate>
  <ray>
    <scan><horizontal>
      <samples>360</samples><resolution>1</resolution>
      <min_angle>-3.14159</min_angle><max_angle>3.14159</max_angle>
    </horizontal></scan>
    <range><min>0.12</min><max>10.0</max><resolution>0.01</resolution>
</range>
    <noise><type>gaussian</type><mean>0.0</mean><stddev>0.01</stddev></noise>
  </ray>
  <plugin name="lidar_plugin" filename="libgazebo_ros_ray_sensor.so">
    <ros><remapping>~/out:=scan</remapping></ros>
    <output_type>sensor_msgs/LaserScan</output_type>
    <frame_name>lidar_link</frame_name>
  </plugin>
</sensor>
```

- type ray is a CPU ray-casting sensor. The alternative gpu_ray is faster but needs a working GPU.
- 360 samples between -pi and +pi give one beam per degree. 10 Hz gives 10 scans per second.
- The range is 0.12 to 10 m, with 1 cm range resolution.
- The noise tag adds Gaussian noise of 1 cm standard deviation to each distance.
- The plugin turns the simulated rays into a sensor_msgs/LaserScan. The remapping ~/out:=scan makes the topic /scan. frame_name must be lidar_link so TF can place the scan on the robot.

21.10 The depth camera

```
<sensor name="depth_camera" type="depth">
  <update_rate>10</update_rate>
  <camera name="depth_cam">
    <horizontal_fov>1.047</horizontal_fov>
    <image><width>160</width><height>120</height><format>R8G8B8</format>
</image>
    <clip><near>0.3</near><far>8.0</far></clip>
  </camera>
  <plugin name="depth_camera_plugin" filename="libgazebo_ros_camera.so">
    <camera_name>camera</camera_name>
    <frame_name>camera_optical_link</frame_name>
    <min_depth>0.3</min_depth><max_depth>8.0</max_depth>
  </plugin>
</sensor>
```

- horizontal_fov 1.047 rad is 60 degrees.
- The image is 160 by 120 pixels at 10 Hz. Small on purpose, because CPU rendering is expensive.
- clip near and far (0.3 and 8 m) is the range that is rendered.
- The plugin publishes /camera/image_raw, /camera/depth/image_raw and /camera/points, using camera_name as the topic prefix. frame_name is the optical frame.

22. slam_sim/worlds/rooms.world

The world is an SDF file (version 1.6). Its structure:

- A physics block: the ODE engine with max_step_size 0.004 s, real_time_update_rate 250 Hz and real_time_factor 1.0. A larger step than the default 0.001 s cuts the CPU load by four.
- Two includes: model://sun (lighting) and model://ground_plane (the floor).
- Many static models. static true means the object does not move and is not simulated as a dynamic body, so it is cheap.

Each wall is one model like this:

```
<model name="wall_n"><static>true</static><pose>0 6 0.75 0 0 0</pose>
  <link name="l">
    <collision name="c"><geometry><box><size>12.2 0.2 1.5</size></box>
</geometry></collision>
    <visual name="v"><geometry><box><size>12.2 0.2 1.5</size></box>
</geometry>
      <material>...colour...</material></visual>
  </link>
</model>
```

The pose gives the centre: x 0, y 6, z 0.75 (half the 1.5 m height, so the box sits on the floor). The collision is what the lidar and the robot touch, and the visual is what is drawn.

The layout:

- Four outer walls: 12.2 m long, 0.2 m thick, 1.5 m tall, centred at x or y of plus or minus 6, so the inner space is 12 by 12 m.
- Two inner walls crossing at the origin, along x = 0 and y = 0. Each is cut into three segments, covering ranges -6 to -3.7, -2.3 to 2.3 and 3.7 to 6. The gaps from -3.7 to -2.3 and from 2.3 to 3.7 are the doorways. They are 1.4 m wide, centred at plus or minus 3.
- Obstacles, 0.8 m high: boxes at (-4.5, 4.5), (3, 4), (-3, -4), (3.5, -4) and a small box at (-1.2, 2.8), and cylinders at (-2, 4.2), (4.8, 2), (-5, -2) and (4.8, -1.8).
- The spawn point (-3, 1.5) is deliberately free of obstacles.

The loop (NW to NE to SE to SW to NW through four doorways) lets slam_toolbox perform loop closure.

23. slam_sim/config/slam_toolbox.yaml

The file starts with the node name slam_toolbox and the key ros__parameters. Groups:

Solver (the graph optimizer):

- solver_plugin: solver_plugins::CeresSolver.
- ceres_linear_solver SPARSE_NORMAL_CHOLESKY, ceres_preconditioner SCHUR_JACOBI, ceres_trust_strategy LEVENBERG_MARQUARDT, ceres_dogleg_type TRADITIONAL_DOGLEG, ceres_loss_function None. These are Ceres numerical settings; the defaults work well.

ROS settings:

- odom_frame odom, map_frame map, base_frame base_footprint.
- scan_topic /scan.
- mode mapping.

- use_map_saver true.
- debug_logging false.
- throttle_scans 1 (use every scan).
- transform_publish_period 0.02 (publish map to odom at 50 Hz).
- map_update_interval 2.0.
- resolution 0.05.
- max_laser_range 10.0.
- minimum_time_interval 0.5.
- transform_timeout 0.2 and tf_buffer_duration 30.0.
- stack_size_to_use 40000000: memory for the solver thread.
- enable_interactive_mode true.

Scan matching and keyframes:

- use_scan_matching true and use_scan_barycenter true.
- minimum_travel_distance 0.2 and minimum_travel_heading 0.2.
- scan_buffer_size 10 and scan_buffer_maximum_scan_distance 10.0: how many recent scans are kept for matching.
- link_match_minimum_response_fine 0.1 and link_scan_maximum_distance 1.5: the conditions for linking scans.

Loop closure:

- do_loop_closing true.
- loop_search_maximum_distance 3.0.
- loop_match_minimum_chain_size 10: how many scans must be in a chain before it can close a loop.
- loop_match_maximum_variance_coarse 3.0, loop_match_minimum_response_coarse 0.35 and loop_match_minimum_response_fine 0.45: how good the match must be.

Correlation (search windows for matching):

- correlation_search_space_dimension 0.5, resolution 0.01, smear deviation 0.1.
- loop_search_space_dimension 8.0, resolution 0.05, smear deviation 0.03.

Penalties (prefer matches close to the prediction):

- distance_variance_penalty 0.5, angle_variance_penalty 1.0.
- fine_search_angle_offset 0.00349, coarse_search_angle_offset 0.349, coarse_angle_resolution 0.0349.
- minimum_angle_penalty 0.9, minimum_distance_penalty 0.5, use_response_expansion true.

24. slam_sim/launch files

24.1 sim.launch.py

It starts the simulation. The main parts:

```
pkg = get_package_share_directory('slam_sim')
robot_description = ParameterValue(
    Command(['xacro ', os.path.join(pkg, 'urdf', 'robot.urdf.xacro')]),
value_type=str)
```

get_package_share_directory finds the installed package folder. Command runs the xacro program on the file at launch time and captures the URDF text. ParameterValue with value_type str stops ROS from trying to parse the text as YAML.

```
gazebo = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(os.path.join(gazebo_ros, 'launch',
'gazebo.launch.py')),
    launch_arguments={'world': world, 'gui': gui}.items())
```

This includes the standard gazebo_ros launch file, which starts the Gazebo server (physics) and client (window) and loads our world. The two launch arguments pass the world path and whether to show the window.

```
rsp = Node(package='robot_state_publisher',
executable='robot_state_publisher',
    parameters=[{'robot_description': robot_description, 'use_sim_time':
True}])
```

robot_state_publisher reads the URDF and publishes all fixed transforms on /tf_static and the robot description on /robot_description (used by RViz2).

```
spawn = Node(package='gazebo_ros', executable='spawn_entity.py',
    arguments=['-entity', 'diff_bot', '-topic', 'robot_description',
               '-x', '-3.0', '-y', '1.5', '-z', '0.05', '-Y', '0.0'])
```

spawn_entity.py reads the URDF from the robot_description topic and inserts it into Gazebo at x -3.0, y 1.5, z 0.05 (slightly above the floor so it settles), with yaw 0 (facing east). This is the spawn pose used everywhere else in the project, including the AMCL start pose.

```
rviz = Node(package='rviz2', executable='rviz2',
    arguments=['-d', os.path.join(pkg, 'rviz', 'slam.rviz')],
    parameters=[{'use_sim_time': True}], condition=IfCondition(use_rviz))
```

RViz2 opens with our saved layout. IfCondition lets you turn it off with rviz:=false.

The file declares the arguments world (default rooms.world), rviz (true) and gui (true), and returns them and the four actions in a LaunchDescription.

24.2 slam.launch.py

```
slam = Node(package='slam_toolbox', executable='async_slam_toolbox_node',
    name='slam_toolbox', output='screen',
    parameters=[os.path.join(pkg, 'config', 'slam_toolbox.yaml'),
{'use_sim_time': True}])
```

It starts the asynchronous SLAM node. The name slam_toolbox must match the top-level key in the YAML file, otherwise the parameters are not applied. Parameters come from the file and from the dictionary that sets simulation time.

24.3 bringup.launch.py

It includes sim.launch.py, and includes slam.launch.py inside a TimerAction with period 8.0 seconds. The delay lets Gazebo, TF and the lidar start first. If SLAM started immediately, it would complain about missing transforms and drop scans. The rviz and gui arguments are passed through to sim.launch.py.

25. slam_sim/rviz/slam.rviz

An RViz2 configuration file (YAML). It stores the displays, tools, camera view and window size so RViz2 opens ready to use.

- Global Options: Fixed Frame map.
- Displays: Grid, RobotModel, TF, Map (topic /map, Durability Transient Local, Reliable), LaserScan (topic /scan, Best Effort, red points), DepthCloud (PointCloud2, /camera/points, disabled by default) and DepthImage (disabled by default).
- View: rviz_default_plugins/TopDownOrtho, a flat top-down orthographic camera, suited to 2D maps. (The class name must be exactly TopDownOrtho.)
- The QoS settings of each display must match the publisher, as explained in Section 2.

26. nav_sim/config/nav2_params.yaml

This one file configures all of Nav2. Each top-level key is the name of a node. Two files exist: nav2_params.yaml (Regulated Pure Pursuit, the default) and nav2_params_dwb.yaml (identical except for the FollowPath block). Every node has use_sim_time True.

26.1 amcl

Localization. Key lines:

```
amcl:
  ros__parameters:
    base_frame_id: 'base_footprint'
    odom_frame_id: 'odom'
    global_frame_id: 'map'
    robot_model_type: 'nav2_amcl::DifferentialMotionModel'
    laser_model_type: 'likelihood_field'
    min_particles: 500
    max_particles: 2000
    max_beams: 60
    update_min_d: 0.25
    update_min_a: 0.2
    set_initial_pose: true
    initial_pose: {x: -3.0, y: 1.5, z: 0.0, yaw: 0.0}
```

- The three frame names must match the TF tree.
- robot_model_type chooses the odometry motion model for a differential-drive robot.
- laser_model_type likelihood_field is the sensor model described in Section 10.3.
- min and max particles bound the adaptive particle count. max_beams 60 uses a subset of the 360 beams to save CPU.
- update_min_d and update_min_a: AMCL only updates after the robot moves 0.25 m or turns 0.2 rad.
- alpha1 to alpha5 (all 0.2): odometry noise for the motion model. sigma_hit 0.2, z_hit 0.5, z_rand 0.5, z_short 0.05, z_max 0.05: the sensor model weights.
- set_initial_pose true plus initial_pose gives the spawn pose, so AMCL starts localized without any click. If this pose differs from where the robot really is, localization starts wrong.
- transform_tolerance 1.0: how far into the future the map to odom transform is stamped; a larger value tolerates a slow simulation.
- tf_broadcast true: AMCL publishes the map to odom transform.

26.2 map_server

```
map_server:
  ros__parameters:
    yaml_filename: ''
```

The file path is left empty on purpose. The launch file passes the real path (the map argument) and the Nav2 launch rewrites this parameter.

26.3 bt_navigator

```
bt_navigator:
  ros__parameters:
    global_frame: map
    robot_base_frame: base_footprint
    odom_topic: /odom
    bt_loop_duration: 10
    default_server_timeout: 20
    plugin_lib_names: [ ... list of BT node libraries ... ]
```

- global_frame and robot_base_frame tell the navigator which frames give the robot pose.
- odom_topic is used to read the robot velocity.
- bt_loop_duration is the tick period of the tree in milliseconds. default_server_timeout is how long it waits for an action server, in milliseconds.
- plugin_lib_names lists the shared libraries that contain the behavior tree nodes (ComputePathToPose, FollowPath, Spin, BackUp, Wait, ClearEntireCostmap, IsStuck, GoalReached, RateController, RecoveryNode, PipelineSequence, RoundRobin and so on). The default tree needs them to load.

26.4 controller_server

```
controller_server:
  ros__parameters:
    controller_frequency: 10.0
    failure_tolerance: 1.0
    progress_checker_plugin: 'progress_checker'
    goal_checker_plugins: ['general_goal_checker']
    controller_plugins: ['FollowPath']
    progress_checker:
      plugin: 'nav2_controller::SimpleProgressChecker'
      required_movement_radius: 0.2
      movement_time_allowance: 20.0
    general_goal_checker:
      plugin: 'nav2_controller::SimpleGoalChecker'
      xy_goal_tolerance: 0.15
      yaw_goal_tolerance: 0.25
      stateful: True
    FollowPath:
      plugin:
'nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController'
      desired_linear_vel: 0.25
      lookahead_dist: 0.6
```

```
      min_lookahead_dist: 0.3
      max_lookahead_dist: 0.9
      lookahead_time: 1.5
      rotate_to_heading_angular_vel: 0.8
      use_rotate_to_heading: true
      rotate_to_heading_min_angle: 0.785
      use_collision_detection: true
      max_allowed_time_to_collision_up_to_carrot: 1.0
      use_regulated_linear_velocity_scaling: true
      regulated_linear_scaling_min_radius: 0.9
      regulated_linear_scaling_min_speed: 0.15
      approach_velocity_scaling_dist: 0.6
```

- The plugin lists are the names of the plugin blocks that follow. The block name FollowPath is the controller identifier that the behavior tree uses; it must stay FollowPath.
- The plugin line picks the algorithm. To switch to DWB you replace the block with plugin dwb_core::DWBLocalPlanner and its parameters (which is what nav2_params_dwb.yaml does).
- The meaning of every RPP parameter is in Section 15.3.
- In the DWB file the FollowPath block has: max_vel_x 0.25, max_vel_theta 0.8, acc_lim_x 1.0, acc_lim_theta 2.5, vx_samples 10, vtheta_samples 20, sim_time 2.0, and the critics list with their scales (BaseObstacle 0.02, PathAlign 32, GoalAlign 24, PathDist 32, GoalDist 24, RotateToGoal 32).

26.5 local_costmap and global_costmap

Both are nested twice, for example local_costmap: local_costmap: ros__parameters. The outer key is the node that owns it, the inner is the costmap itself.

Local costmap:

```
local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 5.0
      publish_frequency: 2.0
      global_frame: odom
      robot_base_frame: base_footprint
      rolling_window: true
      width: 3
      height: 3
      resolution: 0.05
      robot_radius: 0.25
      transform_tolerance: 1.0
      plugins: ['obstacle_layer', 'inflation_layer']
```

- global_frame odom and rolling_window true make a 3 by 3 m window that follows the robot, at 0.05 m resolution.
- robot_radius 0.25 is the footprint circle.
- plugins lists the layers in the order they are applied.

Global costmap: the same, but global_frame map, update_frequency 1.0, track_unknown_space true and plugins static_layer, obstacle_layer, inflation_layer. static_layer has map_subscribe_transient_local True, so it can receive the map that was published once before it started.

The layers:

```
obstacle_layer:
  plugin: 'nav2_costmap_2d::ObstacleLayer'
  observation_sources: scan
  scan:
    topic: /scan
    data_type: 'LaserScan'
    marking: True
    clearing: True
    max_obstacle_height: 2.0
    raytrace_max_range: 3.0
    raytrace_min_range: 0.15
    obstacle_max_range: 2.5
    obstacle_min_range: 0.15
    expected_update_rate: 0.0
inflation_layer:
  plugin: 'nav2_costmap_2d::InflationLayer'
  cost_scaling_factor: 3.0
  inflation_radius: 0.55
```

- marking turns lidar hits into obstacles, clearing removes obstacles along beams that pass through them.
- obstacle_max_range 2.5: only hits closer than 2.5 m are marked. raytrace_max_range 3.0: clearing uses beams up to 3 m.
- The min ranges 0.15 ignore any lidar return closer than 15 cm, in case the robot sees its own parts.
- inflation settings are explained in Section 13.4.

26.6 planner_server

```
planner_server:
  ros__parameters:
    expected_planner_frequency: 20.0
    planner_plugins: ['GridBased']
    GridBased:
      plugin: 'nav2_navfn_planner/NavfnPlanner'
      tolerance: 0.5
      use_astar: False
      allow_unknown: True
```

To use a different global planner you change the plugin line (for example to nav2_smac_planner/SmacPlanner2D) and its parameters.

26.7 smoother_server, behavior_server, waypoint_follower, velocity_smoother

- smoother_server: one plugin, nav2_smoother::SimpleSmoother, with tolerance 1e-10, max_its 1000, do_refinement True.
- behavior_server: behavior_plugins spin, backup, drive_on_heading, wait. costmap_topic local_costmap/costmap_raw and footprint_topic local_costmap/published_footprint let the behaviors check for collisions. global_frame odom. max_rotational_vel 1.0, min_rotational_vel 0.4, rotational_acc_lim 3.2, simulate_ahead_time 2.0.
- waypoint_follower: loop_rate 20, stop_on_failure false, and a task executor WaitAtWaypoint that pauses at each waypoint (200 ms).
- velocity_smoother: smoothing_frequency 20, feedback OPEN_LOOP, max_velocity [0.25, 0.0, 0.8], min_velocity [-0.25, 0.0, -0.8], max_accel [1.0, 0.0, 2.5], max_decel [-1.0, 0.0, -2.5], odom_topic odom, velocity_timeout 1.0. The three numbers are x speed, y speed (0 for a differential robot) and rotation.

27. nav_sim/launch/nav2_sim.launch.py

27.1 Arguments

```
use_slam = LaunchConfiguration('slam')
map_yaml = LaunchConfiguration('map')
params = LaunchConfiguration('params_file')
use_rviz = LaunchConfiguration('rviz')
gui = LaunchConfiguration('gui')
```

LaunchConfiguration objects are placeholders that are filled in with the argument values at launch time. The defaults are declared at the end: slam false, map maps/rooms_map.yaml, params_file config/nav2_params.yaml, rviz true, gui true.

27.2 The simulation

```
sim = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(os.path.join(slam_sim, 'launch',
'sim.launch.py')),
    launch_arguments={'rviz': 'false', 'gui': gui}.items())
```

The whole Gazebo, robot and spawn setup is reused from slam_sim. RViz2 is switched off there (rviz false) because nav_sim opens its own RViz2 with the Nav2 layout. This is why nav_sim depends on slam_sim.

27.3 Nav2 groups

```
nav_args = {'use_sim_time': 'true', 'params_file': params, 'autostart':
'true',
            'use_composition': 'False', 'use_respawn': 'False'}
```

- autostart true: the lifecycle manager activates the nodes by itself.
- use_composition False: each node runs as its own process (easier to debug than one shared process).
- use_respawn False: crashed nodes are not restarted automatically.

```
localization = IncludeLaunchDescription(
    .../nav2_bringup/launch/localization_launch.py,
    launch_arguments={**nav_args, 'map': map_yaml}.items(),
    condition=UnlessCondition(use_slam))
slam = IncludeLaunchDescription(.../slam_sim/launch/slam.launch.py,
    condition=IfCondition(use_slam))
navigation = IncludeLaunchDescription(
    .../nav2_bringup/launch/navigation_launch.py,
    launch_arguments=nav_args.items())
```

- localization_launch.py (from nav2_bringup) starts map_server and amcl with the lifecycle manager. It runs only when slam is false.
- slam.launch.py (from slam_sim) starts slam_toolbox. It runs only when slam is true. In that mode slam_toolbox provides the map and the map to odom transform instead of map_server and AMCL.
- navigation_launch.py starts the planner, controller, smoother, behavior server, bt_navigator, waypoint follower and velocity smoother, with their lifecycle manager. It always runs.
- The Nav2 launch files take the parameter file and rewrite a few values (use_sim_time, the map path) before passing it on.

27.4 Timing

```
nav_stack = TimerAction(period=12.0, actions=[localization, slam,
navigation])

initial_pose = TimerAction(period=22.0, actions=[ExecuteProcess(
    cmd=['ros2', 'topic', 'pub', '-t', '3', '-r', '1', '/initialpose',
         'geometry_msgs/msg/PoseWithCovarianceStamped',
         '{header: {frame_id: map}, pose: {pose: {position: {x: -3.0, y: 1.5,
z: 0.0}, ...}}}'],
    condition=UnlessCondition(use_slam))])
```

- Nav2 starts 12 s after Gazebo, so the robot already exists, and the clock, the lidar and odometry are running.
- At 22 s a command line publishes the start pose three times (-t 3) at 1 Hz (-r 1) on /initialpose. AMCL accepts it as its pose estimate. The covariance gives the uncertainty (0.25 m2 for x and y, 0.0685 rad2 for yaw). This is a safety net in case the initial_pose parameter was not applied in time.
- It only runs in non-SLAM mode.

27.5 RViz2

```
rviz = Node(package='rviz2', executable='rviz2', output='screen',
    arguments=['-d', os.path.join(pkg, 'rviz', 'nav.rviz')],
    parameters=[{'use_sim_time': True}], condition=IfCondition(use_rviz))
```

The function returns the arguments, sim, nav_stack, initial_pose and rviz in one LaunchDescription.

28. nav_sim/scripts/send_goals.py

A small Python program that sends the robot to four goals in turn, using the Simple Commander API.

```
GOALS = [('NE room', 3.0, 2.0, 0.0),
         ('SE room', 2.0, -2.0, math.pi),
         ('SW room', -3.0, -2.0, math.pi / 2),
         ('NW room (start)', -3.0, 1.5, 0.0)]
```

Each goal is a name, x, y and yaw in the map frame. They are placed in free space in each room and in an order that forces the robot to pass through the doorways.

```
def make_pose(nav, x, y, yaw):
    p = PoseStamped()
    p.header.frame_id = 'map'
    p.header.stamp = nav.get_clock().now().to_msg()
    p.pose.position.x, p.pose.position.y = x, y
    p.pose.orientation.z = math.sin(yaw / 2)
    p.pose.orientation.w = math.cos(yaw / 2)
    return p
```

A ROS pose stores orientation as a quaternion. For a rotation by yaw only about the z axis, the quaternion is (x 0, y 0, z sin(yaw/2), w cos(yaw/2)). The header says the pose is in the map frame, with the current simulation time.

```
rclpy.init()
nav = BasicNavigator()
if '--slam' in sys.argv:
    nav._waitForNodeToActivate('bt_navigator')
else:
    nav.waitUntilNav2Active()
```

BasicNavigator is the wrapper around the Nav2 actions. waitUntilNav2Active waits until AMCL and the navigator are active. In SLAM mode there is no AMCL to wait for, so the script waits for bt_navigator only (the --slam flag).

```
for name, x, y, yaw in GOALS:
    nav.goToPose(make_pose(nav, x, y, yaw))
    while not nav.isTaskComplete():
        fb = nav.getFeedback()
        ...distance_remaining logged every 3 s...
    result = nav.getResult()
    if result == TaskResult.SUCCEEDED: ...
    else: ...log error and stop...
```

goToPose sends the NavigateToPose action goal. The loop polls isTaskComplete, reads feedback such as the remaining distance, and then checks the result (SUCCEEDED, CANCELED or FAILED). If a goal fails, the tour stops.

29. nav_sim/tools/generate_map.py and the map files

Why generate a map: the navigation stage should work without first driving around to map. The map is computed from the same numbers as the world file, so it matches exactly.

Settings: RES 0.05 m per cell, SIZE_M 12.2, ORIGIN (-6.1, -6.1), giving N = 243 cells per side. The values written to the image: FREE 254, OCC 0.

The script lists the same rectangles (walls and boxes, each as centre x, centre y, size x, size y) and circles (centre and radius) as rooms.world. For every pixel it does:

```
x = ORIGIN[0] + (c + .5) * RES
y = ORIGIN[1] + (N - r - .5) * RES
```

This converts a pixel (row r, column c) into the world coordinates of its centre. The N - r part flips the vertical axis, because row 0 of an image is the top, and the top of the map is the largest y. The pixel is marked occupied (0) if it lies inside any rectangle or circle, otherwise free (254).

The image is written in the binary PGM (P5) format: a text header with the width, height and 255, then one byte per pixel. The YAML file gets image, mode trinary, resolution 0.05, origin [-6.1, -6.1, 0.0], negate 0, occupied_thresh 0.65 and free_thresh 0.25 (Section 7.3).

If you change the world, edit the lists at the top of the script and run python3 tools/generate_map.py.

30. nav_sim/rviz/nav.rviz

The RViz2 layout for navigation. Fixed frame map, top-down orthographic view (TopDownOrtho), and a Navigation 2 panel. Displays:

- Map: /map (Transient Local, because map_server publishes it once).
- Global Costmap: /global_costmap/costmap, alpha 0.35, colour scheme costmap.
- Local Costmap: /local_costmap/costmap, alpha 0.5.
- LaserScan: /scan, red points, Best Effort.
- Global Path (green): /plan, line width 0.06 m. This is the line from the robot to the goal.
- Local Path (red): /local_plan, line width 0.04 m (published by DWB).
- Footprint: /local_costmap/published_footprint, a magenta outline.
- RobotModel: /robot_description.

Tools: MoveCamera, Select, SetInitialPose (2D Pose Estimate, publishes /initialpose) and SetGoal (2D Goal Pose, publishes /goal_pose). The costmap and path displays use Volatile durability, which works with any publisher.

31. How to change things

- Make the robot bigger or smaller: edit body_l, body_w and wheel properties in robot.urdf.xacro, keep wheel_sep above body_w and update robot_radius in both costmaps.
- Change the robot speed: nav2_params.yaml, desired_linear_vel (RPP), max_velocity of velocity_smoother, and the controller limits.
- Add an obstacle: add a model to rooms.world, and the same shape in generate_map.py, then regenerate the map.
- Change the start position: the -x, -y, -Y arguments in sim.launch.py, amcl initial_pose in nav2_params.yaml and the initial_pose message in nav2_sim.launch.py.
- Smoother global paths: change the planner plugin to nav2_smac_planner/SmacPlanner2D (install ros-humble-nav2-smac-planner).
- Use DWB: launch with params_file pointing to nav2_params_dwb.yaml.
- Closer passing of walls: lower inflation_radius or raise cost_scaling_factor.
- Faster, lighter simulation: gui:=false, lower sensor update rates and image size in the URDF, increase max_step_size in the world.
- A different world: copy rooms.world, edit it, launch sim.launch.py with world:=/path/to/file.world, and regenerate or re-map.

- Use the depth camera for obstacles: install depthimage_to_laserscan, convert /camera/depth/image_raw into a scan and add it as a second observation source (observation_sources: scan camera_scan) in both obstacle layers.

32. Command cheat sheet

```
# Build
cd ~/slam_ws && colcon build --symlink-install && source install/setup.bash

# Stage 1: mapping
ros2 launch slam_sim bringup.launch.py
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -p speed:=0.3
-p turn:=0.8
ros2 run nav2_map_server map_saver_cli -f ~/rooms_map

# Stage 2: navigation on the known map
ros2 launch nav_sim nav2_sim.launch.py
ros2 run nav_sim send_goals.py

# Navigation while mapping
ros2 launch nav_sim nav2_sim.launch.py slam:=true

# DWB instead of Pure Pursuit
ros2 launch nav_sim nav2_sim.launch.py
params_file:=$HOME/slam_ws/src/nav_sim/config/nav2_params_dwb.yaml

# Checks
ros2 lifecycle get /bt_navigator
ros2 run tf2_ros tf2_echo map odom
ros2 topic hz /scan
```